CSE 242 Assignment 2

3 Questions, 100 pts

Your name: Youngseo Cho
Student ID: 2346292

Instruction

- Submit your assignments onto **Canvas** by the due date. Upload a `zip` file containing:

  (1) The saved/latest `.ipynb` file.

  (2) Also save your file into a pdf version, if error appears, save an html version instead (easy to grade for written questions).

  (3) All other materials to make your `.ipynb` file runnable.

  For assignment related questions, please reach TA or grader through EdDiscussion.

- This is an **individual** assignment. All help from others (from the web, books other than text, genAI, or people other than the TA or instructor) must be clearly acknowledged.
- Most coding parts can be finished with only 1-2 lines of codes.
- Make sure you have installed required packages: `scikit-learn`.
- Double click to edit each markdown cell.

Objective

- **Task 1:** Maximum likelihood estiamtion (math)
- **Task 2:** Linear Regression (with Scikit-learn)
- **Task 3:** Principle Component Analysis (with Scikit-learn)

Question 1 (Maximum likelihood estiamtion and Posterior, 20 pts)

Assume we have a coin that has some unknown probability h of coming up heads (and probability $1-h$ of coming up tails). If the coin is flipped five times getting three heads and two tails ($\mathbf{HHHTT}$) then:

(a -- 10pts) What is the maximum likelihood estimate for h?

(b -- 10pts) Assume that (before flipping the coin) we have a prior density $p(h)$ for the various values of $h \in [0,1]$. Give the formula for the posterior probability density $p(h \mid \mathbf{HHHTT})$ as a function of the prior $p(h)$.

For Question 1(b), the solution is acceptable if the formula contains an integral.

Solution (a): First, we can calculate the likelihood of $\mathbf{HHHTT}$ as : $f(h) = h^3(1-h)^2$.

Now, to maximize likelihood, differentiate $f(h) \rightarrow$
$\frac{df(h)}{dh} = 3h^2(1-h)^2 - 2h^3(1-h)$

$$\begin{aligned}\frac{df(h)}{dh} &= 3h^2(1-h)^2 - 2h^3(1-h) \\ &= h^2(1-h)(3-5h)\end{aligned}$$

Setting the derivative to zero:

$$h^2(1-h)(3-5h) = 0$$

The critical points are $h = 0$, $h = 1$, and $h = \frac{3}{5}$.

Since $f(0) = f(1) = 0$ and $f(h) > 0$, the maximum likelihood estimate is:

$$\hat{h}_{\text{MLE}} = \frac{3}{5} = 0.6$$

Solution (b): Using Bayes' rule:

$$p(h \mid \mathbf{HHHTT}) = \frac{p(\mathbf{HHHTT} \mid h)p(h)}{p(\mathbf{HHHTT})}$$

The likelihood is:

$$p(\mathbf{HHHTT} \mid h) = h^3(1-h)^2$$

The denominator is the normalization constant, obtained by integrating over all possible values of h:

$$p(\mathbf{HHHTT}) = \int_0^1 u^3(1-u)^2 p(u), du$$

Therefore, the posterior probability density is:

$$p(h \mid \mathbf{HHHTT}) = \frac{h^3(1-h)^2p(h)}{\int_0^1 u^3(1-u)^2p(u)du}$$

If you are not familair with Latex, you may attach a figure/screen-shoot and display the code below.

In [51]:
```
from IPython.display import Image
# Replace the figure name
# Image(filename='sample.png')
```

Question 2 (Linear Regression, 40 pts)

In this question, you will be using **Scikit-learn** to empirically apply the Linear Regression model. We will adopt an advertisement dataset **Advertising.csv** from Kaggle.

Reading data using Pandas

In [52]:
```
# Read the dataset you will be working on
# The dataframe loaded with pandas is named as data

import pandas as pd
data = pd.read_csv('Advertising.csv', index_col=0)  # Modify your d

# Take a look at the first 5 rows
data.head()
```

Out[52]:

	TV	Radio	Newspaper	Sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

In [53]:
```
data.shape
```

Out[53]:
```
(200, 4)
```

Features and responses

What are the features?

- **TV:** advertising dollars spent on TV for a single product in a given market (in thousands of dollars)
- **Radio:** advertising dollars spent on Radio
- **Newspaper:** advertising dollars spent on Newspaper

What is the response?

- **Sales:** sales of a single product in a given market (in thousands of items)

What else do we know?

- There are 200 **observations**, and each observation (each row) is a single market.
- **Your main task**: predict the sales of a single product in a given market.
- Since the response variable is continuous, this is a **regression** problem.

Linear regression

Pros: fast, no tuning required, highly interpretable

Cons: adopting the linear regression is unlikely to always generate the best predictive accuracy, since Linear Regression presumes a linear relationship between the features and response. This assumption may not always make sense. (But if one can *transform* the features so that do have a linear relationship, sometimes progress can be made, for example by taking logs.)

Expression:

- y: the response
- β_0: the intercept
- β_1: the coefficient for x_1 (the first feature)
- β_i is the coefficient for x_i (the ith feature, $i \in \{1, 2, \ldots, n\}$)

For this regression task:

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

The β values ($\beta = [\beta_0, \beta_1, \beta_2, \beta_3]$) are called the **model coefficients**. These values are "learned" during the model fitting step using the "least squares" criterion. Then, we can make use of the fitted model to make predictions!

Prepare the dataset for training use

- Scikit-learn expects X (feature matrix) and y (response vector) to be NumPy arrays.

- Pandas is built on top of NumPy, and exposes numpy methods. Thus, X can be a pandas DataFrame, y can be a pandas Series!

In [54]:
```
# Prepare the feature X
X = data[['TV', 'Radio', 'Newspaper']]

# check the type and shape of X
print(type(X))
print(X.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
(200, 3)
```

In [55]:
```
# Prepare the response Y
y = data['Sales']
# check the type and shape of y
print(type(y))
print(y.shape)
```

```
<class 'pandas.core.series.Series'>
(200,)
```

Question 2.1 (Split the dataset into train and test parts, 5 pts)

For features X and response y, use **sklearn** to perform the the splitting of dataset: 80% for train data, 20% for the test data.

In [56]:
```
################## Your answer for Question 2.1   ##################
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=

################## Your code above #################
```

Double check the shape of train and test data:

In [57]:
```
# default split is 80% for training and 20% for testing
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

Question 2.2 (Train a Linear regression model via Scikit-learn, 10 pts)

Assume you model is named as **model**, and you train the linear regression

model to fit on the training data.

In [58]:
```
################# Your answer for Question 2.2   #################
# Name your linear regression model as 'model', so proceeding cells

from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

################# Your code above #################
```

Out[58]:
```
LinearRegression()
```

In [59]:
```
# Take a look at your model coefficients
import numpy as np
print(np.round(model.intercept_, 4))
print([np.round(val, 4) for val in model.coef_])
```

```
2.9791
[0.0447, 0.1892, 0.0028]
```

In [60]:
```
# Match the feature names with the trained coefficients
list(zip(data.columns[:3].tolist(), model.coef_))
```

Out[60]:
```
[('TV', 0.04472951746871633),
 ('Radio', 0.18919505423437655),
 ('Newspaper', 0.002761114341367238)]
```

Question 2.3 (Interpreting model coefficients, 10 pts)

The coefficients of each feature are printed above.

Your tasks:

- What is the trained linear regression model?
- **(replace β_i below with the trained coefficients, reserve four decimal places, 5 pts)**

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

- **(Open) question (5 pts)**: how do you interpret the coefficient of **TV** (β_1)?

Your solution for Question 2.3:

The output above shows that

$\beta_0 = 2.9791, \beta_1 = 0.0447, \beta_2 = 0.1892, \beta_3 = 0.0028$. Thus,

$$\text{Sales} = 2.9791 + 0.0447 \times \text{TV} + 0.1892 \times \text{Radio} + 0.0028 \times \text{Newspaper}$$

Interpretation of the coefficient of TV

The coefficient of TV (β_1) refers to the expected change of increasing sales for a one-unit increase of TV advertising. Assuming other advertisements (Radio and Newspaper) are constant, an additional $\$1,000$ spent on TV advertising is associated with an expected increase of ($\beta_1 \times 1,000 = 44.7$) units in sales.

Making predictions

In [61]:
```
# make predictions on the testing set
y_pred = model.predict(X_test)
# Print the test accuracy score from your trained linear regression
model.score(X_test,y_test)
```

Out[61]:
```
0.899438024100912
```

Model evaluation metrics for regression

Evaluation metrics for classification problems, such as **accuracy**, are not useful for regression problems. Instead, we need evaluation metrics designed for comparing continuous values. (Since predictions would hardly ever exactly match the measured quantities.)

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is a popular evaluation metric for regression, which is defined as the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Question 2.4 (Evaluate your predictions on test features with RMSE metric, 5 pts)

Hint: you may calculate MSE using scikit-learn, but RMSE is a little bit different from MSE.

In [62]:
```
from sklearn import metrics
################# Your answer for Question 2.4   #################

y_pred = model.predict(X_test)
rsme = np.sqrt(metrics.mean_squared_error(y_test, y_pred))

print("RMSE: ", rsme)

################# Your code above #################
```

```
RMSE:  1.7815996615334506
```

Question 2.5 (Does the feature 'Newspaper' help? 10 pts)

Does the feature **Newspaper** contribute or impair the model prediction? Now repeat previous procedure:

- For features X, only select **TV** and **Radio**; response y is unchanged.
- Split the dataset as what you have done before.
- Fit/Train the model on the train data.
- Make predictions on the test data.
- Compute the RMSE of the predictions on the test data.
- **Show your observations!**

Reminder: if you use `random_state` while splitting the dataset, make sure the value of `random_state` is the same for the two splits. So that the test data only differs in the column of **Newspaper**.

In [63]:
```
################# Your answer for Question 2.5   #################
X_reduced = data[['TV', 'Radio']]
X_train_reduced, X_test_reduced, y_train_reduced, y_test_reduced =


model_reduced = LinearRegression()
model_reduced.fit(X_train_reduced, y_train_reduced)

y_pred_reduced = model_reduced.predict(X_test_reduced)
rsme_reduced = np.sqrt(metrics.mean_squared_error(y_test_reduced, y

print("RMSE without Newspaper: ", rsme_reduced)

################# Your code above #################
```

```
RMSE without Newspaper:  1.7714254173033517
```

Your observations:

Using all three features (TV, Radio, and Newspaper), the test RMSE is approximately 1.7816. When Newspaper is removed and the model is retrained on the same train/test split, the test RMSE decreases slightly to approximately 1.7714. Thus, in this particular split, excluding Newspaper slightly improves test predictions. Newspaper does not appear to provide additional predictive value here, although the small difference alone does not establish that it is never useful on other datasets or splits.

Question 3 (PCA, 40 pts)

In this question, we will explore how PCA helps with reducing the dimensionality through scikit-learn. We will adopt a simple digits dataset contained in scikit-learn.

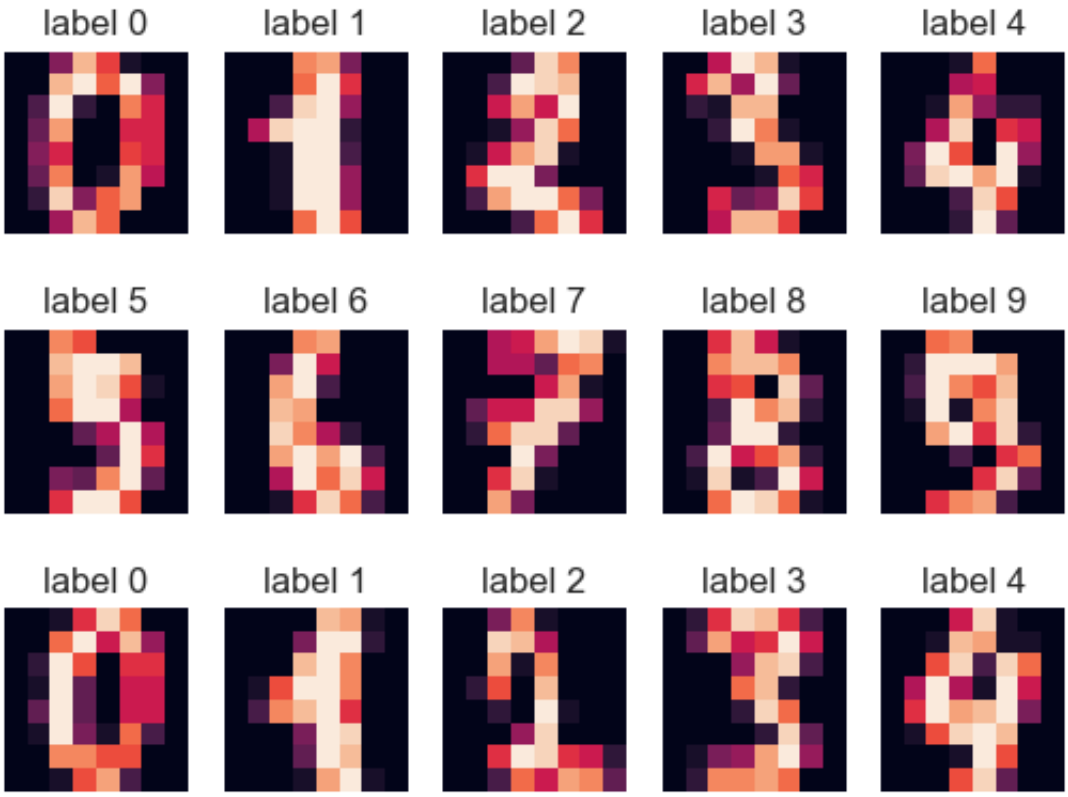
In [64]:
```
# import libraries
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns;
sns.set()
```

In [65]:
```
from sklearn.datasets import load_digits
data_digit = load_digits()
data_digit.data.shape
```

Out[65]:
```
(1797, 64)
```

As you have seen above, there are 1797 rows and 64 columns in this dataset. Each row denotes the pixel values of an image. Since each image is 8*8 (in pixel), the dimension of each feature is 64. Let's take a look at a few images contained in this dataset.

In [66]:
```
# A help function to display images with labels
def display_digits(imgs):
    h = 3
    w = 5
    fig, axes = plt.subplots(h, w)
    for i in range(h):
        for j in range(w):
            if imgs.shape[1] == 64:
                axes[i, j].imshow(imgs[i * w + j].reshape(8, 8))
            else:
                axes[i, j].imshow(imgs[i * w + j])
            axes[i, j].axis('off')
            axes[i, j].set_title(f'label {data_digit.target[i * w +
# Take a look at a few examples
display_digits(data_digit.images)
```



PCA finds uncorrelated orthogonal axes (principal components) in a high dimensional space (i.e., 64) to project the feature onto principal components.

Question 3.1 (PCA and dimensionality deduction with Scikit-learn, 10 pts)

In this question, you may use the implementation of PCA from scikit-learn, as imported below. Detailed explanations of parameters settings are available in this link.

Your task in this question:

Fit the model with feature X (**data_digit.data** for this dataset) and apply the dimensionality reduction on X, keep 2 components.

In [67]:
```
from sklearn.decomposition import PCA

################## Your answer for Question 3.1   ##################

# name the transformed (dimension reduced) feature as projected
projected = PCA(n_components=2).fit_transform(data_digit.data)
################## Complete the code above ##################


# The code below checks whether the feature dimension 64 is reduced
print(data_digit.data.shape)
print(projected.shape)
```

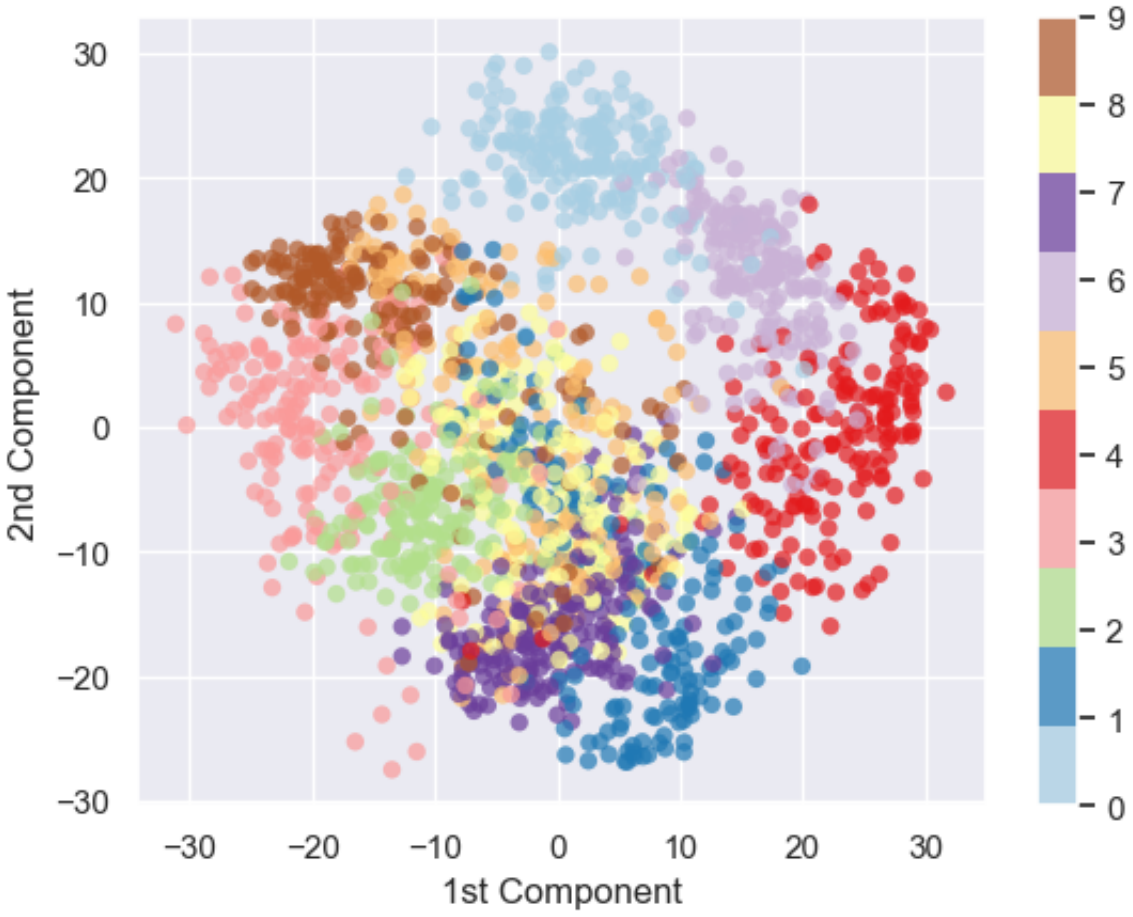
```
(1797, 64)
(1797, 2)
```

With the reduced dimension of features, we can now visualize the projected/reduced features in a 2-dimension plot as below. There are hardly any overlaps between digit 0 and any other digits. While in the central area of the figure, features are not well separable.

In [68]:
```
plt.scatter(projected[:, 0], projected[:, 1],
            c=data_digit.target, edgecolor='none', alpha=0.7,
            cmap=plt.cm.get_cmap('Paired', 10))
plt.xlabel('1st Component')
plt.ylabel('2nd Component')
plt.colorbar()
plt.figure(figsize=(18, 8))
```

```
/var/folders/_4/s97pfzsx5vd79cc18fsmcybw0000gn/T/ipykernel_8964/284023998.py:3: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed two minor releases later. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)`` instead.
  cmap=plt.cm.get_cmap('Paired', 10))
```

Out[68]: <Figure size 1800x800 with 0 Axes>



<Figure size 1800x800 with 0 Axes>

Question 3.2 (How many components do we

need? 15 pts)

In practice, suppose you are given a high-dimension dataset, and you are interested in performing a dimension reduciton with PCA. How many conponents do we need? One method is to look at the distribution of the explained variance ratio w.r.t. each component.

In this question, you may use the function `explained_variance_ratio` in **pca** (with scikit-learn) to address this question. Detailed explanations of parameters settings are available in this link.

Your task in this question:

Print the ratio of variance explained by each of the selected components;

Visualize with matplotlib or seaborn (x axis: $i-th$ component; y axis: the corresponding ratio of explained variance).

In [69]:
```
################## Your answer for Question 3.2   #################

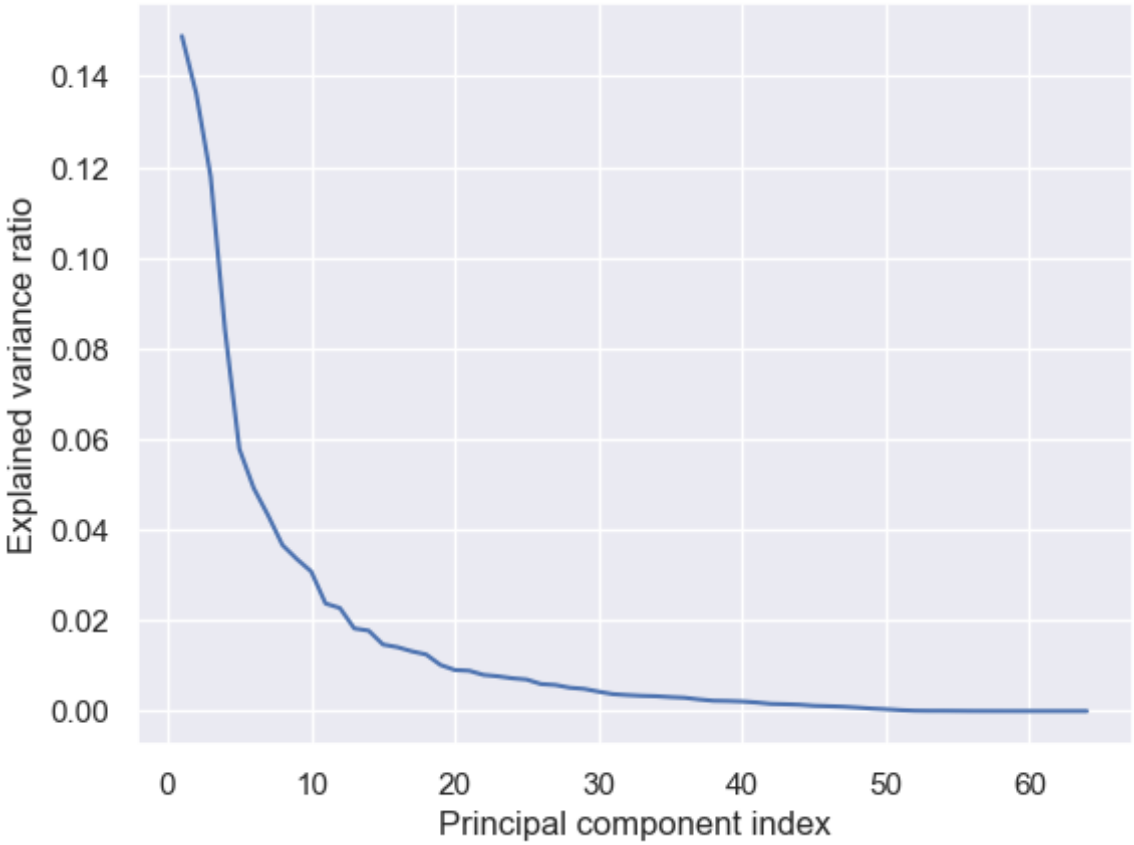

pca_full = PCA().fit(data_digit.data)
variance_ratios = pca_full.explained_variance_ratio_

print(np.round(variance_ratios, 4))

plt.plot(
    np.arange(1, len(variance_ratios) + 1),
    variance_ratios
)
plt.xlabel('Principal component index')
plt.ylabel('Explained variance ratio')
plt.show()

################## Complete the code above ################
```

```
[1.489e-01 1.362e-01 1.179e-01 8.410e-02 5.780e-02 4.920e-02 4.320e-
02
 3.660e-02 3.350e-02 3.080e-02 2.370e-02 2.270e-02 1.820e-02 1.770e-
02
 1.470e-02 1.410e-02 1.320e-02 1.250e-02 1.020e-02 9.100e-03 8.900e-
03
 8.000e-03 7.700e-03 7.200e-03 7.000e-03 6.000e-03 5.800e-03 5.200e-
03
 4.900e-03 4.300e-03 3.700e-03 3.500e-03 3.400e-03 3.300e-03 3.100e-
03
 2.900e-03 2.600e-03 2.300e-03 2.200e-03 2.100e-03 1.900e-03 1.600e-
03
 1.500e-03 1.400e-03 1.200e-03 1.100e-03 1.000e-03 8.000e-04 6.000e-
04
 4.000e-04 2.000e-04 1.000e-04 1.000e-04 1.000e-04 0.000e+00 0.000e+
00
 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+
00
 0.000e+00]
```



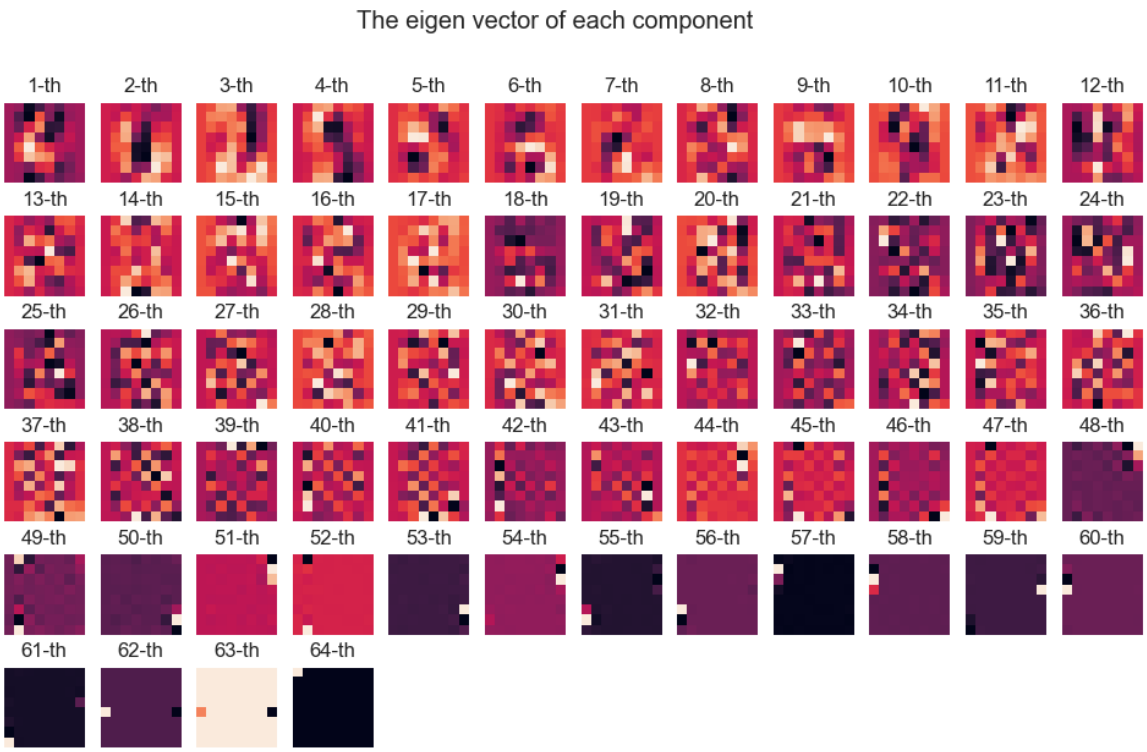
How to obtain eigen values and vectors from sklearn PCA

- Eigen values -- `your-PCA-model.explained_variance_`
- Eigen vectors -- `your-PCA-model.components_`

Eigen vectors (Eigen digits)

In [70]:
```
# Default # of components is the dimension of feature space
pca = PCA()
pca.fit(data_digit.data)
h = 6
w = 12
fig, axes = plt.subplots(h, w)
fig.set_size_inches(12, 7)
for i in range(h):
    for j in range(w):
        idx = i * w + j
        if idx > 63:
            axes[i, j].axis('off')
        else:
            axes[i, j].imshow(pca.components_[idx].reshape(8, 8))
            axes[i, j].axis('off')
            axes[i, j].set_title(f'{idx + 1}-th')
fig.suptitle('The eigen vector of each component')
```

Out[70]: `Text(0.5, 0.98, 'The eigen vector of each component')`



Question 3.3 What are your observations from the visualized eigen vectors? (5 pts)

Your solutions for Question 3.3:

The front eigenvectors show relatively clear, structured patterns resembling the common strokes and shapes found in handwritten digits. These principal components capture directions of unique characteristics among the digit images, rather than representing individual digits themselves. In later components, the patterns generally become more localized and less

interpretable, including weak variation around background pixels. Because PCA orders components by explained variance, the first components capture more of the dataset's variation than later ones. However, a low-variance component is not necessarily pure noise.

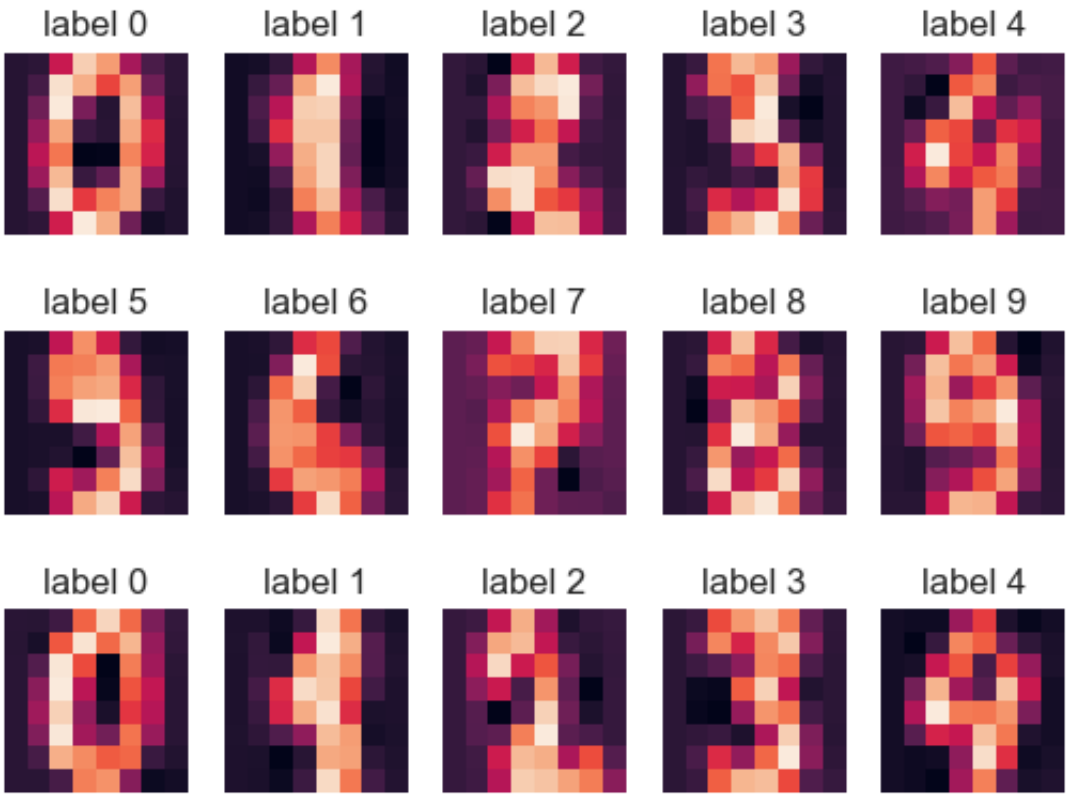
Feature reconstruction:

Suppose you only want to preserve 80% variance after applying PCA for the dimension reduction, due to storage issues, etc.

In [71]:
```
# Use pca from sklearn to fit on the feature space
pca_08 = PCA(0.8).fit(data_digit.data)

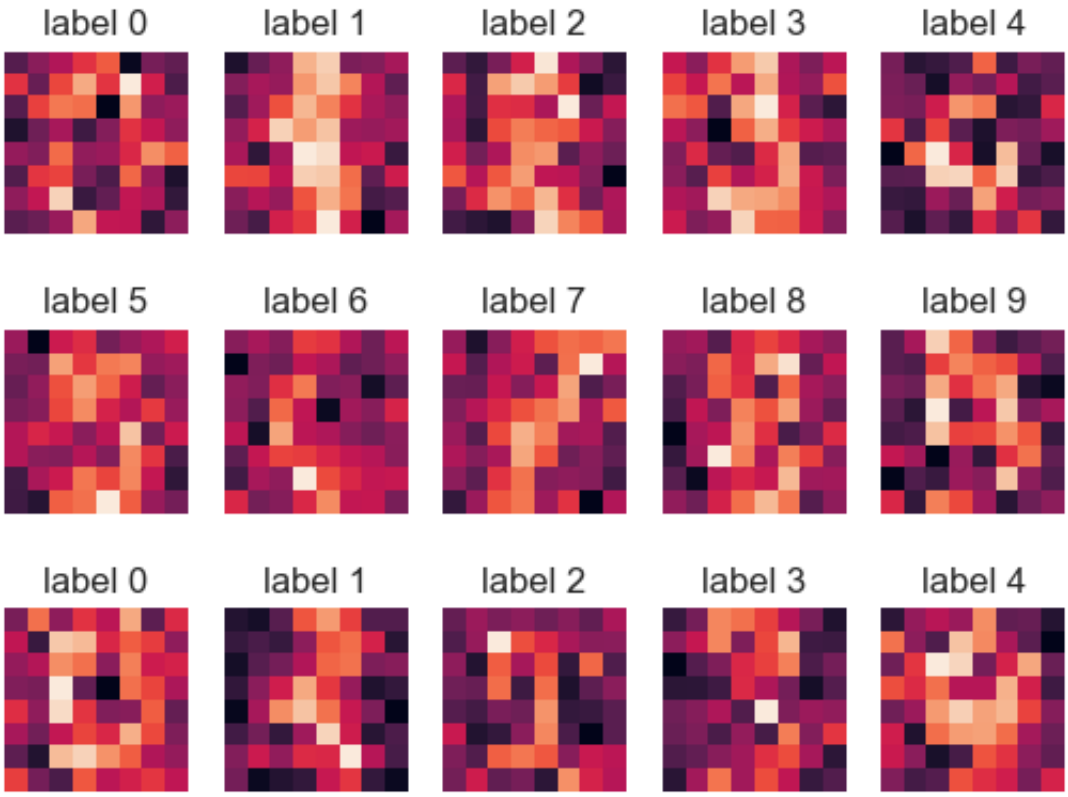
# Reconstruct
lower_dimension = pca_08.fit_transform(data_digit.data)
reconstructed = pca_08.inverse_transform(lower_dimension)

# Display the reconstructed images (with a much lower dimension)
display_digits(reconstructed)
```



Assume now you are given only a set of perturbed images (with some random noise). As shown below, the figures become much more difficult to recognize.

In [72]:
```
# Apply random noise on clean images
noisy_imgs = np.random.normal(data_digit.data, 5)
# Display the noisy images
display_digits(noisy_imgs)
```



Question 3.4 Does PCA mitigate the random noise? (10 pts)

Your tasks:

- Perform feature reconstruction on `noisy_imgs` (preserving 80% variance after applying PCA for the dimension reduction)
- Explain your understandings on why PCA reconstructs cleaner images.

In [73]:
```
################## Your answer for Question 3.4   ##################



# name the reconstructed images from noisy images as 'reconstructed

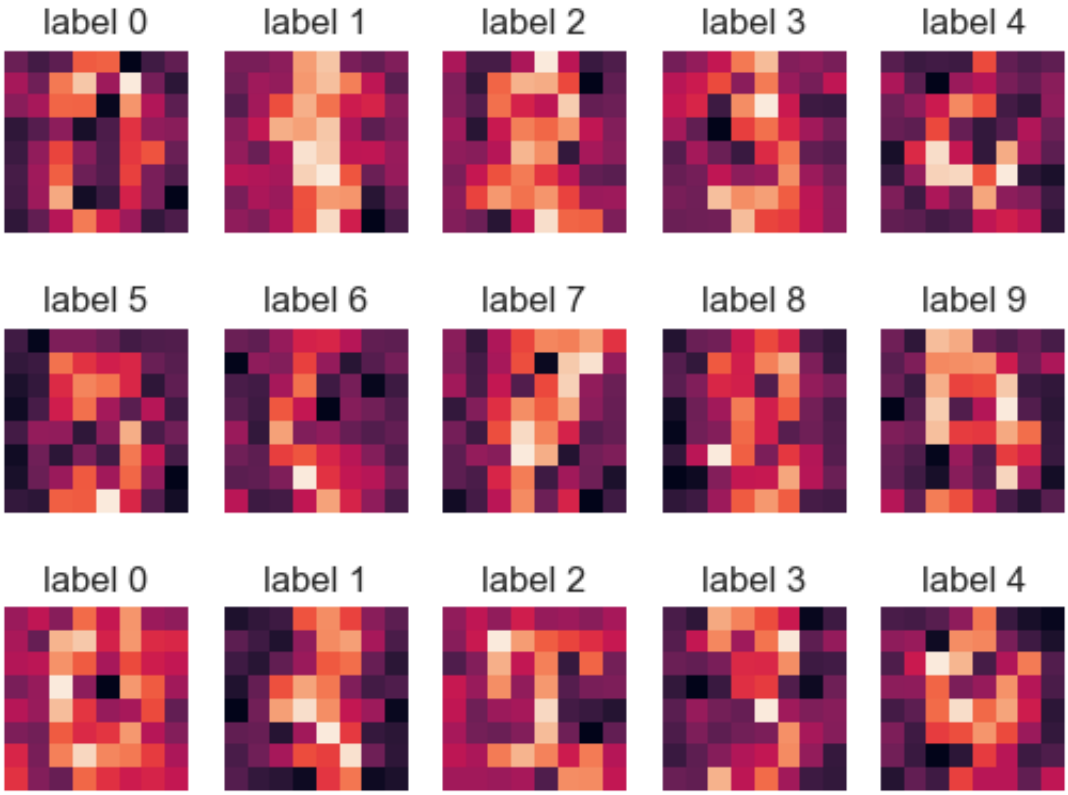
pca_noisy = PCA(n_components=0.8)

compressed_noisy = pca_noisy.fit_transform(noisy_imgs)
reconstructed_noise = pca_noisy.inverse_transform(compressed_noisy)

print("Components:", pca_noisy.n_components_)
print("Noisy MSE:", np.mean((noisy_imgs - data_digit.data)**2))
print("Reconstructed MSE:", np.mean((reconstructed_noise - data_dig

# Display the reconstructed images
display_digits(reconstructed_noise)
################## Complete the code above ##################
```

```
Components: 40
Noisy MSE: 24.960620077422913
Reconstructed MSE: 17.427317012858595
```



Your written answers for Question 3.4:

PCA can partially reduce random noise by projecting images onto the leading principal components and reconstructing them from a lower-dimensional representation. Some noise lies in discarded lower-variance directions, so the reconstructed digits may look smoother. In this experiment, PCA fitted on the noisy images retained at least 80% of their variance. Thus, the pixel-wise MSE relative to clean images decreased from 25.18 to 17.94. However, PCA maximizes explained variance rather than explicitly identifying noise, so some noise may remain and useful fine details can also be lost.